
Project Overview: Automated Weekly Newsletter Generator (Group Project)

1. Project Description

For this capstone, we built an automated weekly newsletter generator for Kamau, the Content Lead at “Nairobi Nexus”. The workflow runs every Friday at 2:00 PM (Africa/Nairobi), fetches raw HTML content from five Kenyan news sources (TechCabal, Business Daily, The Star, The Standard, Kenyan Wallstreet), and uses a two-stage AI process. First, five dedicated Field Reporter AI agents each extract the single most important headline from their assigned source and output clean JSON. Second, a Chief Editor AI agent synthesises all five headlines into a short, witty newsletter introduction that uses Kenyan slang, is formatted in Markdown, and stays under 200 words. The final draft is appended to a Google Sheet (“NewsletterManager”, tab “Drafts”) and emailed to the marketing team for approval.

A separate Error Workflow (`Newsletter Error Handler`) is linked to the main workflow and is activated only when the Chief Editor node fails (e.g., timeout, invalid parameter). It generates a simpler fallback draft and sends an alert.

2. Problem Solved

Before this automation, Kamau spent hours manually skimming multiple Kenyan publications, collecting top stories, and drafting a weekly newsletter from scratch. The process was slow, inconsistent, and often missed important headlines. We solved this by:

- Scraping five sources in parallel, reducing total execution time.
- Using dedicated AI agents to extract headlines at the source, ensuring clean, structured data.
- Automatically generating a consistent, engaging newsletter intro that follows brand voice constraints (slang, Markdown, length).
- Building a fault-tolerant architecture: if one source fails, the others continue; if the Chief Editor fails, a dedicated error workflow produces a simpler draft.
- Logging every draft and the number of successful sources for auditability.

3. Workflow Architecture (n8n)

We used n8n Cloud to build two workflows:

- Main Workflow (Automated Weekly Newsletter Generator Final – production version; also a Testing Copy used for validating error handling)
- Error Workflow (Newsletter Error Handler)

3.1 Main Workflow Nodes (Production)

Node Name	Type	Purpose
Schedule Trigger (or Webhook for testing)	Schedule / Webhook	Triggers workflow on Friday 14:00 (production) or via webhook for testing.
HTTP * 5	HTTP Request	Fetches raw HTML from each source. All HTTP nodes have <code>onError: continueErrorOutput</code> and a 10s timeout.
AI Agent * 5 (Field Reporters)	AI Agent (LangChain)	Each extracts one headline. Outputs JSON <code>{"source": "...", "headline": "..."}.</code> Their error handling is also set to <code>continueErrorOutput</code> .

Structured Output Parser * 5	Output Parser	Enforces the JSON schema on each Field Reporter.
Merge	Merge (Append, 5 inputs)	Collects all successful headline objects as separate items.
Aggregate	Aggregate (<code>aggregateAllItemData</code>)	Combines all items into a single array under key <code>data</code> .
AI Agent_Chief Editor	AI Agent (LangChain)	Synthesises headlines into a newsletter intro with Kenyan slang, Markdown, under 200 words. Outputs <code>{"intro": "..."}.</code>
Structured Output Parser5	Output Parser	Enforces the <code>intro</code> schema.
Append row in sheet	Google Sheets (Append)	Writes the draft to the "Drafts" tab of the <code>NewsletterManager</code> sheet. Columns: Date, Successful Sources, Draft Body, Model Used.
Send a message	Gmail (Send)	Emails the final draft to the marketing team.

3.2 Testing Copy – Forcing Chief Editor Failure

We created a testing copy of the main workflow (`Automated Weekly Newsletter Generator Testing Copy.json`) to validate that the error workflow is triggered specifically when the Chief Editor fails. In this copy:

- The Chief Editor node uses only one chat model: `OpenAI Chat Model` with `llama-3.3-70b-versatile`.
- Maximum tokens is set to `0` (zero).
- Sampling temperature is `0.1`.
- Timeout is set to `100` milliseconds.

These settings guarantee that the Chief Editor node fails immediately (zero tokens are invalid, and the timeout is impossibly short). Consequently, the main workflow's execution ends with an `Error` status, and the linked error workflow is activated.

3.3 Error Workflow (Newsletter Error Handler)

This workflow is linked to the main workflow via the `errorWorkflow` setting. It is triggered only when the main workflow fails – which, in our test, happens exclusively when the Chief Editor node fails (other nodes continue on error). The error workflow consists of:

Node Name	Type	Purpose
Error Trigger	Error Trigger	Listens for failures from the main workflow.
AI Agent	AI Agent (LangChain)	Generates a very short, simple fallback intro (under 100 words) using basic Kenyan slang.
Structured Output Parser	Output Parser	Enforces <code>{"intro": "..."} </code> schema.

Send a message	Gmail	Sends the fallback draft to the marketing team with an explanatory note.
----------------	-------	--

Because the error workflow does not receive the original headlines – only the error metadata – the fallback draft is generic (e.g., “Sasa! We’re having a small tech issue. Full newsletter next week.”).

4. AI Agent 1 – Field Reporters (One per Source)

All Field Reporter nodes are identical except for the source name. Below is the configuration for TechCabal; the others follow the same pattern.

4.1 User Message (Expression mode)

```
text
={{ $json.data }}
```

(Note: `$json.data` comes from the HTTP node’s output.

4.2 System Message (example for TechCabal)

```
text
You are a field reporter for Nairobi Nexus.
```

Analyze the provided HTML and extract the single most important tech or business headline.

IMPORTANT:

- Return ONLY valid JSON
- Do not include explanations
- Do not use markdown
- Do not wrap the JSON in backticks

Correct format:

```
{
  "source": "TechCabal",
  "headline": "headline text here"
```

```
}
```

4.3 Structured Output Parser – JSON Schema

```
json
```

```
{  
  "type": "object",  
  "properties": {  
    "source": { "type": "string" },  
    "headline": { "type": "string" }  
  },  
  "required": ["source", "headline"]  
}
```

5. AI Agent 2 – Chief Editor (Newsletter Synthesis)

5.1 User Message (Expression mode)

```
text
```

Here are the headlines from Kenyan publications:

```
{{ JSON.stringify($json.data) }}
```

Respond with ONLY this JSON and nothing else – no markdown fences, no explanation:

```
{"intro": "your markdown newsletter here"}
```

5.2 System Message

```
text
```

You are Kamau, a witty tech editor in Nairobi. Synthesize the following headlines into a short, engaging newsletter intro.

Constraints:

1. Strictly under 200 words.
2. Use Kenyan slang (e.g., "Bazeng", "Form ni gani", "Sasa", "Niaje").
3. Format the output as Markdown.

Output ONLY the newsletter intro, no extra text.

5.3 Structured Output Parser – JSON Schema

```
json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "intro": {
      "type": "string",
      "description": "The newsletter introduction text, strictly under 200 words, using Kenyan slang, formatted as Markdown."
    }
  },
  "required": ["intro"],
  "additionalProperties": false
}
```

The AI outputs `{"output": {"intro": "..."}} (the output wrapper is added automatically by n8n). Therefore the Google Sheets expression uses {{ $json.output.intro }}.`

6. Error Handling & Model Fallback

6.1 Fault Tolerance in Field Reporter Branches

Every HTTP node is configured with `onError: continueErrorOutput` and a timeout of 10 seconds. Additionally, all Field Reporter AI agents have `onError: continueErrorOutput`. This ensures that if one source fails (e.g., website down, timeout), the other four branches continue to execute, and the Merge node only receives successful headlines.

6.2 Error Workflow – Triggered Only by Chief Editor Failure

The main workflow's settings include `errorWorkflow: "qNsi3wyWD1B5q5uQ"` (the ID of the Newsletter Error Handler). The error workflow contains an Error Trigger that captures any failure of the main workflow. However, because all other nodes are set to `continueErrorOutput`, the only way the main workflow can enter an Error state is if the Chief Editor node fails. We verified this using the testing copy where the Chief Editor is forced to fail via `maxTokens=0` and a 100ms timeout.

When the Chief Editor fails, the error workflow runs:

- Its AI Agent uses a simplified prompt (under 100 words, basic Kenyan slang, apology).
- A structured output parser enforces `{"intro": "..."}.`
- The Gmail node sends the fallback draft to the marketing team.

7. Google Sheets – Drafts Tab (NewsletterManager)

The `Append row in sheet` node writes to the `Drafts` tab of the `NewsletterManager` spreadsheet. Column mappings:

Sheet Column	n8n Expression
Date	<code>=={{ \$now }}</code>
Successful Sources	<code>=={{ \$node["Aggregate"].json.data.length }}</code>
Draft Body	<code>=={{ \$json.output.intro }}</code>
Model Used	<code>mistral-large-latest(fallback:mistral-large-latest) (static)</code>

The `Successful Sources` count is the length of the array produced by the `Aggregate` node – i.e., the number of sources that returned a valid headline.

8. Email Briefing – The Executive Briefing

The Gmail node (Send a message) sends the final newsletter draft to the marketing team. The subject and body are dynamic:

- Subject: Nairobi Nexus Draft – `{{ $now.toFormat('yyyy-MM-dd') }}`
- Body (HTML/Text): `={{ $json["Draft Body"] }}`

The `$json` refers to the output of the Google Sheets append node, which returns the written row. This ensures the email contains exactly what was logged.

9. Testing & Validation

We performed two types of tests:

9.1 Normal Production Run

We triggered the main workflow using the production webhook URL (simulating an automatic schedule). All five HTTP requests completed (some with timeouts, but `continueOnErrorOutput` allowed the rest to proceed). Each Field Reporter output valid JSON, the Aggregate node produced an array, and the Chief Editor generated a newsletter intro under 200 words containing Kenyan slang and proper Markdown. A row was appended to the Google Sheet, and an email was received with the draft.

9.2 Forced Chief Editor Failure (Error Workflow Test)

We used the testing copy where the Chief Editor's `maxTokens` was set to 0 and timeout to 100ms. When we triggered this workflow via webhook, the Chief Editor node failed immediately. The main workflow's execution status became `Error`. The linked error workflow was automatically triggered, and we received an email containing a very short, simple fallback intro (e.g., "Sasa! Our tech team hit a glitch. Full stories next week."). This confirmed that the error workflow activates exclusively on Chief Editor failure and produces a compliant simpler draft.

After validation, we reverted the testing copy and published the production workflow with the correct schedule (Friday 14:00 Africa/Nairobi).

10. Conclusion

We successfully built a robust, parallel-processing, fault-tolerant weekly newsletter generator that meets all rubric requirements:

- Distributed parallel architecture – five independent HTTP → AI Agent branches before merging.
- Fault tolerance – `Continue on Error` on all HTTP and Field Reporter nodes.
- Model fallback – dedicated error workflow triggered only when the Chief Editor fails, generating a simpler draft.
- Prompt engineering & output quality – distinct roles (Field Reporters vs. Chief Editor), Kenyan slang, Markdown, under 200 words.
- Logging & delivery – Google Sheets append to `NewsletterManager` (Drafts tab) and Gmail email with dynamic content.

Kamau can now rely on an automated, consistent, and resilient newsletter generation system. The workflow handles source failures gracefully and even provides a fallback draft when the main AI editor encounters problems.